DSP MINI PROJECT REPORT

ON

" Interpolation for Smooth Reconstruction of Sensor Signals "

Under the Course

DIGITAL SIGNAL PROCESSING

SUBMITTED BY

PRN	NAME OF STUDENT
1022512003	Rushikesh Gajanan Borade

2026-27

Third Year Electronics and Communication Engineering (TY ECE)

Division:B , Semester - I

Under the Guidance Of
Dr. Hrishikesh Vanjari

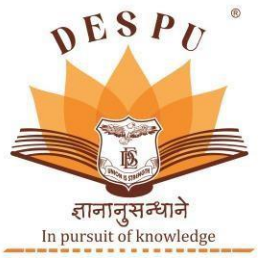
Department of Electronics and Communication
School of Engineering and Technology
DES Pune University, Pune.

2026-2027

Program: B. Tech Electronics and Communication (AI/ML)Engineering

School of Engineering and Technology

DES Pune University, Pune



CERTIFICATE

This is to certify that student **Rushikesh Gajanan Borade** , PRN.**1022512003** is studying in **TY ECE (Division:B)** course in SEM I and he has successfully completed and submitted the **DSP Mini Report** under the Course **"Digital Signal Processing"** , entitled **"Interpolation for Smooth Reconstruction of Sensor Signals"** .This study is a partial fulfillment of the degree of Bachelor of Technology in Electronics and Communication Engineering of **DES Pune University, Pune** during the academic year **2026-2027.**

Dr. Hrishikesh Vanjari — Mentor

Dr. Prachi Joshi — HoS

Acknowledgement

We would like to express our sincere gratitude to our faculty mentor, Dr. Hrishikesh Vanjari, for his valuable guidance, support, and encouragement throughout the completion of this Digital Signal Processing mini-project titled “Interpolation for Smooth Reconstruction of Sensor Signals.”

We would also like to thank the Department of Computer Engineering and Technology, School of Engineering and Technology, and DES Pune University, Pune, for providing the opportunity and academic environment to undertake this project.

We are grateful to the developers of MATLAB and its documentation, and to the authors of the research papers, the LibriSpeech corpus and the SciPy library listed in the references, for providing the tools and technical resources required to implement decimation, interpolation and FIR filtering and to evaluate the experimental results using RMSE, MAE and correlation.

This project has helped us strengthen our understanding of sampling, aliasing, multirate signal processing, interpolation filter design, and the practical application of Digital Signal Processing concepts to sensor data.

Abstract

Sensors often deliver sparse measurements, either because bandwidth, storage or power is limited or because the sampling rate is deliberately reduced. This project studies how accurately such signals can be reconstructed by interpolation after downsampling. We built one reproducible MATLAB benchmark in which a signal is optionally anti-alias filtered, downsampled by a factor M, and reconstructed with five methods: zero-order hold, linear interpolation, cubic spline, FFT-based (sinc) interpolation and polyphase FIR interpolation (the multirate method of Crochiere and Rabiner). Quality is measured with RMSE, MAE and correlation on a synthetic multi-tone sensor signal with noise and on a real audio recording. Linear interpolation is used as the existing baseline.

At M = 4 on the synthetic signal, polyphase FIR gave an RMSE of 0.0424 against 0.0495 for linear and 0.146 for zero-order hold. Without noise, spline and polyphase interpolation were about 15 to 18 times more accurate than linear (0.0014 and 0.0017 against 0.025). The largest factor keeping RMSE below 0.10 was M = 8 for spline, FFT and polyphase, M = 5 for linear and M = 2 for zero-order hold; error rose sharply at M = 10, where the new Nyquist frequency equals the highest tone (50 Hz). Pre-filtering helped at M = 4 but not at every factor. On real audio at M = 8, polyphase FIR reached an RMSE of 0.0168 and correlation of 0.983 against 0.0289 and 0.967 for linear (42% lower RMSE). The code, tables and figures form a reusable reconstruction benchmark.

Keywords: interpolation, decimation, multirate DSP, polyphase FIR filter, cubic spline, aliasing, RMSE, MAE, MATLAB

INDEX

Chapter No.	Topic	Page No.
1	**Introduction**	**7**
1.1	Background	7
1.2	Need/Importance	7
1.3	Objectives	7
1.4	Scope	8
2	**Literature Review**	**9**
2.1	Relevant concepts, theories	9
2.2	Technologies related to the project	10
2.3	Summary of similar works or case studies	10
3	**Proposed Methodology**	**11**
3.1	System Overview	11
3.2	Architecture / Workflow Diagram	11
3.3	Data Collection / Input Description	11
3.4	Algorithms / Techniques Used	11
3.5	Tools, software, or equipment used	12
3.6	Module-wise Implementation Plan	12

Chapter No.	Topic	Page No.
3.7	Validation and Testing Strategy	12
4	**Results / Observations**	**14**
4.1	Experimental / Implementation Setup	14
4.2	Performance Metrics	14
4.3	Comparison with Expected Results	14
4.4	Observations and Analysis	15
4.5	Validation and Verification with summary of findings	20
5	**Future Enhancement**	**21**
6	**Conclusion & Recommendations**	**22**
7	**References**	**23**

1 Introduction

1.1 Background

Sensors such as temperature probes, accelerometers, ECG electrodes and microphones measure a continuous physical quantity at discrete instants. In practice the stored or transmitted data can be sparse: devices reduce the sampling rate to save energy, memory and bandwidth, and measurements may be missing or coarsely spaced. Multirate digital signal processing provides the tools to change the sampling rate of a signal. Decimation lowers the rate, and interpolation raises it again by estimating the values between the available samples. How well the original waveform is recovered depends on the interpolation method, the downsampling factor and the filtering used before and after the rate change.

1.2 Need/Importance

Analysis, visualisation and control algorithms usually expect a smooth, densely sampled signal. A poor interpolator introduces staircase or corner artefacts and distorts the spectrum, while an unsuitable downsampling factor causes aliasing that cannot be repaired afterwards. Engineers therefore need quantitative guidance on which interpolation approach to use, how much data reduction is acceptable, and whether pre-filtering is worthwhile. A side-by-side benchmark on identical data answers these questions with numbers rather than intuition.

1.3 Objectives

1. Implement a downsample-then-interpolate pipeline for sensor signals in MATLAB.
2. Compare five interpolation approaches: zero-order hold, linear, cubic spline, FFT-based (sinc) and polyphase FIR interpolation.
3. Determine the optimal downsampling/interpolation factor for a given signal bandwidth.
4. Study the effect of pre-filtering (anti-aliasing) and post-filtering (anti-imaging).
5. Evaluate reconstruction quality quantitatively using RMSE, MAE and correlation on synthetic and real data.
6. Compare the proposed approach with an existing method: linear interpolation as the common baseline, and MATLAB’s built-in multirate functions based on the Crochiere and Rabiner polyphase theory.

1.4 Scope

The project covers single-channel, uniformly sampled signals, integer factors from M = 2 to 20, and offline evaluation in MATLAB. A synthetic multi-tone signal with controllable noise and a real audio recording are used. Non-uniform sampling, fractional rate conversion, multichannel data and real-time hardware implementation are outside the scope.

2 Literature Review

2.1 Relevant concepts, theories

Sampling theorem. A signal whose highest frequency is fmax can be recovered exactly from samples taken at a rate fs greater than 2·fmax. Frequencies above fs/2 fold back (alias) into the band and cannot be separated from true content.

Downsampling (decimation) by M. The low-rate signal is xd[n] = x[nM], with new rate fs/M. To avoid aliasing, x is first passed through a low-pass anti-aliasing filter with cutoff near fs/(2M). This is the pre-filter.

Upsampling and imaging. Upsampling by L inserts L − 1 zeros between samples. This creates spectral images centred at multiples of the low sampling rate. A low-pass anti-imaging filter (the post-filter) with cutoff π/L and gain L removes the images and fills the zeros with interpolated values.

Interpolation methods compared.

1. *Zero-order hold:* repeats the last sample; piecewise constant, a staircase waveform with strong spectral images.
2. *Linear:* straight lines between neighbouring samples (a triangular kernel); continuous but with corners; error proportional to the square of the sample spacing.
3. *Cubic spline:* piecewise cubic polynomials with continuous first and second derivatives; smooth, with error proportional to the fourth power of the spacing.
4. *FFT (sinc) interpolation:* zero-pads the spectrum, equivalent to ideal band-limited (sinc) interpolation under a periodic assumption.
5. *Polyphase FIR interpolation:* upsampling followed by a low-pass FIR anti-imaging filter, implemented as L short polyphase sub-filters so that no multiplications by inserted zeros are performed (Crochiere and Rabiner).

Performance metrics. For original x and reconstruction $\hat{x}$ over N samples:

- RMSE = $\mathrm{sqrt}\left((1/N) \Sigma (x[n] - \hat{x}[n])^2 \right)$
- MAE = $(1/N) \Sigma |x[n] - \hat{x}[n]|$
- Correlation $\rho = \mathrm{cov}(x, \hat{x}) / (\sigma x \cdot \sigma \hat{x})$

RMSE penalises large errors, MAE gives the average error magnitude, and correlation measures similarity of shape irrespective of scale.

Noise floor. When the reference contains white noise of standard deviation σ and the pre-filter removes the noise above the new Nyquist frequency, no interpolator can restore that removed noise. The expected error floor is approximately σ·sqrt(1 − 1/M).

2.2 Technologies related to the project

1. **MATLAB** with the Signal Processing Toolbox: decimate (FIR pre-filtering and downsampling), resample (polyphase FIR rate conversion), interpft (FFT interpolation) and interp1 (zero-order hold, linear and spline interpolation).
2. **SciPy** (open-source Python library): resample_poly and resample provide the polyphase and FFT methods used here; it serves as a reference implementation of the same algorithms.
3. **LibriSpeech** open speech corpus as a source of real recorded signals.

2.3 Summary of similar works or case studies

- *Crochiere and Rabiner, Multirate Digital Signal Processing (Signal Processing, Elsevier, 1983)*, the reference supplied by our guide: the standard treatment of decimation, interpolation and efficient polyphase filter structures. It is the basis of the polyphase method and the "existing method" used for comparison.
- *Schafer and Rabiner (1973)* presented interpolation as upsampling followed by low-pass filtering, the principle behind the FIR interpolators used here.
- *Oppenheim and Schafer, Discrete-Time Signal Processing*: standard textbook source for sampling, aliasing and rate conversion theory.
- *SciPy documentation*: describes FFT-based and polyphase resampling routines that mirror MATLAB's interpft and resample.
- *Reference from guide (PubMed ID 40534299)*: [add a one or two sentence summary of this paper after reading its abstract].

Most of these works explain the theory of a single method. This project adds a side-by-side quantitative comparison of several methods under one protocol, on both synthetic and real data.

3 Proposed Methodology

3.1 System Overview

The benchmark takes a reference signal, optionally applies an anti-aliasing filter, keeps every M-th sample, reconstructs the original rate with each interpolation method, and scores the result against the reference. The same signals, factors and metrics are used for every method so the comparison is fair.

3.2 Architecture / Workflow Diagram

Input signal x[n] → Pre-filter (optional anti-aliasing FIR low-pass) → Downsample by M → Low-rate samples xd → Interpolator (zero-order hold / linear / cubic spline / FFT / polyphase FIR) → Reconstructed signal $\hat{x}$[n] → Edge trimming → Metrics (RMSE, MAE, correlation) → Tables and figures.

[Insert here: block diagram of the workflow, drawn from the flow above.] Block diagram of the proposed downsample-and-interpolate workflow.

3.3 Data Collection / Input Description

Synthetic sensor signal. Sampling rate fs = 1000 Hz, duration 2 s (N = 2000 samples), made of three sinusoids at 5 Hz (amplitude 1.0), 20 Hz (0.5) and 50 Hz (0.25), plus white Gaussian noise of standard deviation 0.05 (random seed fixed with rng(0)). Noise levels of 0, 0.05, 0.1 and 0.3 were used in the noise-robustness test. The signal is saved as data/synthetic_sensor.csv.

Real signal. [State the file you used: a 2-second excerpt of a LibriSpeech FLAC recording, or MATLAB's built-in handel audio, and give its sampling rate.] The excerpt is the real-data input for Experiment 6.

3.4 Algorithms / Techniques Used

1. **Pre-filter and downsampling:** MATLAB decimate with the FIR option (low-pass FIR of order 30·M with cutoff at the new Nyquist frequency) or plain sample dropping x(1:M:end) when pre-filtering is disabled.
2. **Zero-order hold:** interp1 with the previous-value rule.
3. **Linear interpolation:** interp1 with the linear rule.
4. **Cubic spline:** interp1 with the spline rule.
5. **FFT (sinc) interpolation:** interpft, zero-padding the spectrum.
6. **Polyphase FIR interpolation:** resample(xd, M, 1), an anti-imaging FIR filter applied through a polyphase structure.

7. **Scoring:** RMSE, MAE and correlation after trimming a few samples at both ends to avoid filter edge effects.

3.5 Tools, software, or equipment used

MATLAB (MATLAB Online) with the Signal Processing Toolbox; the LibriSpeech corpus; the SciPy library as reference; a standard computer with a web browser. No special hardware was needed.

3.6 Module-wise Implementation Plan

Module	Function	Purpose
Signal generation	script section	Create and save the synthetic signal
Downsampling	downsample_sig	Pre-filter (optional) and keep every M-th sample
Interpolation	reconstruct	Rebuild the original rate with the chosen method
Edge handling	trim_edges	Remove filter edge effects before scoring
Metrics	m_rmse, m_mae, m_corr	Quantitative evaluation
Spectrum plot	plot_spec	Show aliasing with and without pre-filter
Experiments 1 to 6	script sections	Method comparison, factor study, pre-filter study, noise test, spectrum, real data

3.7 Validation and Testing Strategy

- Every method is run on identical data, factors and metrics.
- The random seed is fixed so results are repeatable.
- A noise-free test shows whether an interpolator is accurate on its own, without noise masking the differences.

- Results are checked against theory: the expected error floor $\sigma \cdot \sqrt{1 - 1/M}$ and the Nyquist limit at $f_s/M = 2 \cdot f_{max}$.
- Independent implementations of the band-limited method (FFT and polyphase) should agree closely.
- Conclusions from synthetic data are checked on a real recording.

4 Results / Observations

4.1 Experimental / Implementation Setup

Experiments were run in MATLAB with the Signal Processing Toolbox [state your MATLAB release]. Factors tested: M = 2, 3, 4, 5, 8, 10, 15, 20 (synthetic) and M = 2, 4, 8 (real audio). Fifty to sixty samples were trimmed from each end of the synthetic signal and 300 from the audio signal before scoring.

4.2 Performance Metrics

RMSE, MAE and correlation coefficient, as defined in Section 2.1. Lower RMSE and MAE and higher correlation indicate better reconstruction.

4.3 Comparison with Expected Results

Expectation from theory	Observed	Agrees
Zero-order hold is the worst method	RMSE 0.146 against 0.042 to 0.050 for the others (M = 4)	Yes
Smooth interpolators beat linear	Polyphase 0.0424 against linear 0.0495 (about 14% lower)	Yes
Error rises sharply when fs/M falls to 2·fmax	RMSE jumps from 0.072 (M = 8) to 0.187 (M = 10)	Yes
Noise floor of about σ·sqrt(1 − 1/M)	Expected 0.043 at M = 4, observed 0.0424	Yes
Pre-filtering always reduces error	Helps at M = 4; not at M = 8	Partly
Synthetic conclusions hold on real data	Same method ranking on real audio	Yes

4.4 Observations and Analysis

Experiment 1: method comparison (M = 4, synthetic).

Method	RMSE	MAE	Correlation
Zero-order hold	0.14607	0.1045	0.98402
Linear	0.049505	0.039087	0.9982
Cubic spline	0.042447	0.033607	0.99865
FFT (sinc)	0.04264	0.033794	0.99864
Polyphase FIR	0.042399	0.033613	0.99865

Zero-order hold gives a visible staircase and by far the largest error. Linear interpolation is about three times better. Spline, FFT and polyphase are practically identical and about 14% better than linear. Their RMSE of about 0.042 is close to the noise floor, because the reference contains noise that the anti-aliasing filter correctly removes and no method can restore.

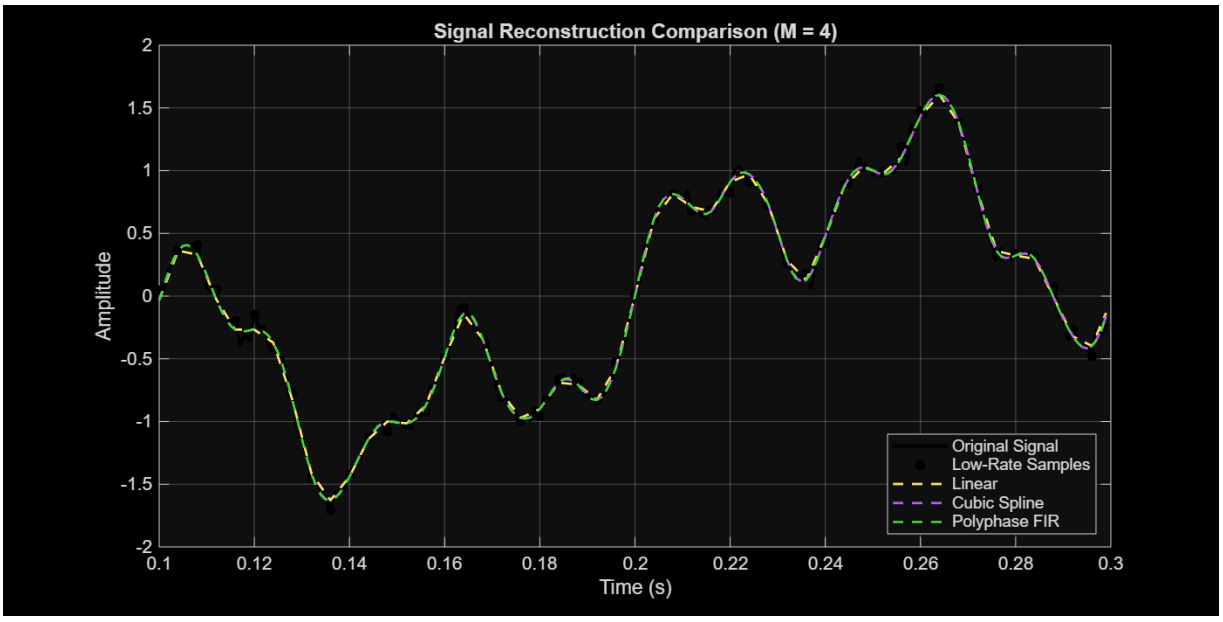


Figure 1: Signal Reconstruction Comparison (M = 4). The original signal, the low-rate samples and the linear, cubic spline and polyphase FIR reconstructions over a zoomed time window.

Experiment 2: RMSE against downsampling factor M (synthetic).

M	Zero-order hold	Linear	Cubic spline	FFT (sinc)	Polyphase FIR
2	0.0655	0.0353	0.0338	0.0336	0.0336
3	0.1051	0.0432	0.0394	0.0392	0.0391
4	0.1462	0.0496	0.0425	0.0427	0.0425
5	0.1834	0.0603	0.0443	0.0442	0.0441
8	0.2754	0.1286	0.0924	0.0728	0.0724
10	0.3132	0.1990	0.1872	0.1868	0.1869
15	0.4129	0.2355	0.2041	0.2012	0.2012
20	0.4970	0.2813	0.2274	0.1994	0.1989

The largest factor with RMSE below 0.10 was M = 8 for spline, FFT and polyphase, M = 5 for linear and M = 2 for zero-order hold. Better interpolators therefore allow stronger data reduction for the same quality. Error grows slowly up to M = 5 and then jumps at M = 10. At M = 10 the low sampling rate is 100 Hz, so the new Nyquist frequency equals the highest tone (50 Hz) and that component is lost; the extra error (about 0.18 RMSE) matches the RMS of the 50 Hz tone ($0.25/\sqrt{2} = 0.177$). The practical rule is to keep fs/M clearly above 2·fmax; M = 8 (fs/M = 125 Hz, 2.5 times fmax) worked.

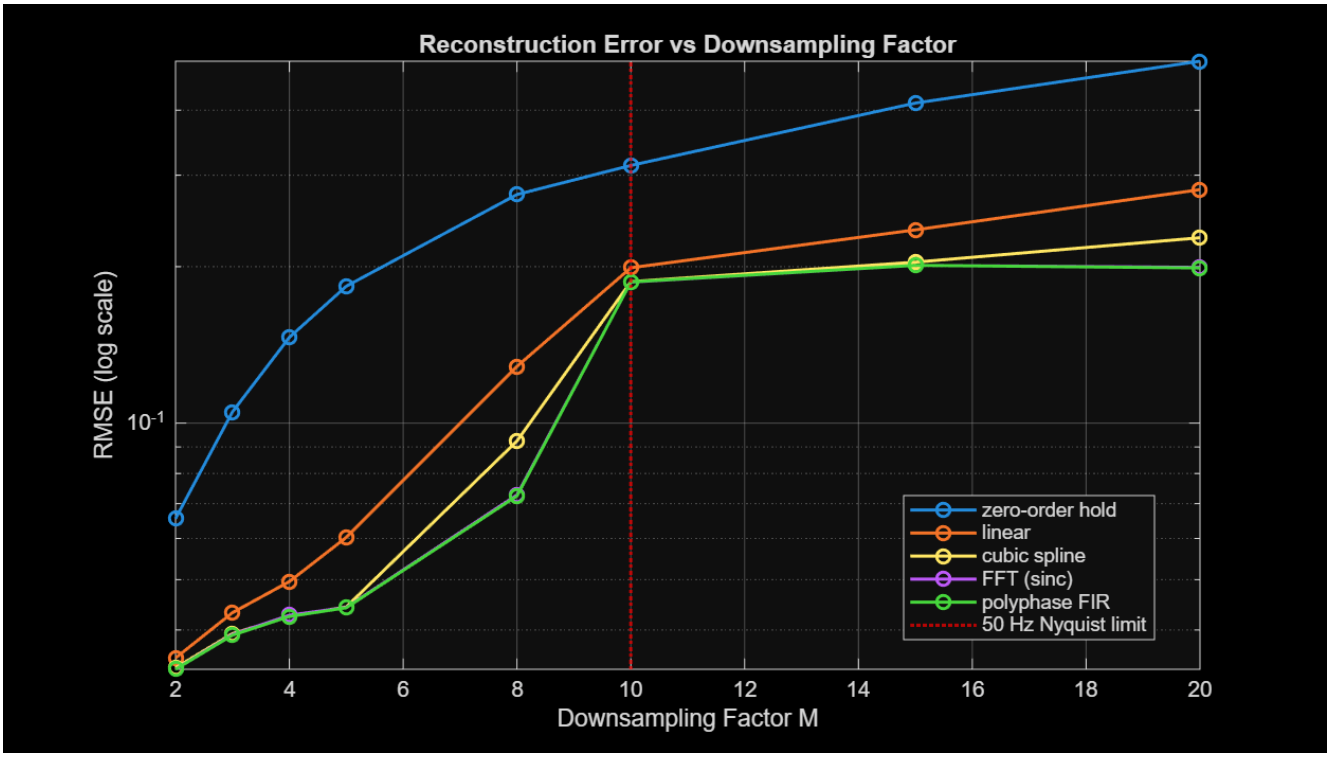


Figure 2: Reconstruction Error vs Downsampling Factor. RMSE of the five methods for M = 2 to 20, with the Nyquist limit of the 50 Hz tone marked.

Experiment 3: effect of pre-filtering (RMSE, synthetic).

M	Method	With pre-filter	Without pre-filter
4	Linear	0.0496	0.0583
4	Cubic spline	0.0425	0.0573
4	Polyphase FIR	0.0425	0.0591
8	Linear	0.1286	0.1113
8	Cubic spline	0.0924	0.0763
8	Polyphase FIR	0.0724	0.0646
12	Linear	0.2025	0.1943
12	Cubic spline	0.1901	0.2207
12	Polyphase FIR	0.2033	0.2620

Pre-filtering clearly helps at M = 4, where it removes out-of-band noise. At M = 12 it helps the spline and polyphase methods, because the 50 Hz tone lies above the new Nyquist frequency (41.7 Hz) and without a filter it folds down to about 33 Hz, which the downsampled spectrum figure shows as a false peak. At M = 8, however, the unfiltered signals gave slightly lower error. All tones are below the new Nyquist frequency there, so there is little true signal aliasing to prevent, and the long pre-filter (order 30·M) can cost accuracy through its transition band and through start-up transients near the signal edges, which grow with M. We therefore conclude that pre-filtering is beneficial when content above the new Nyquist frequency exists, but is not automatically beneficial, and that its design (cutoff, length, edge handling) matters.

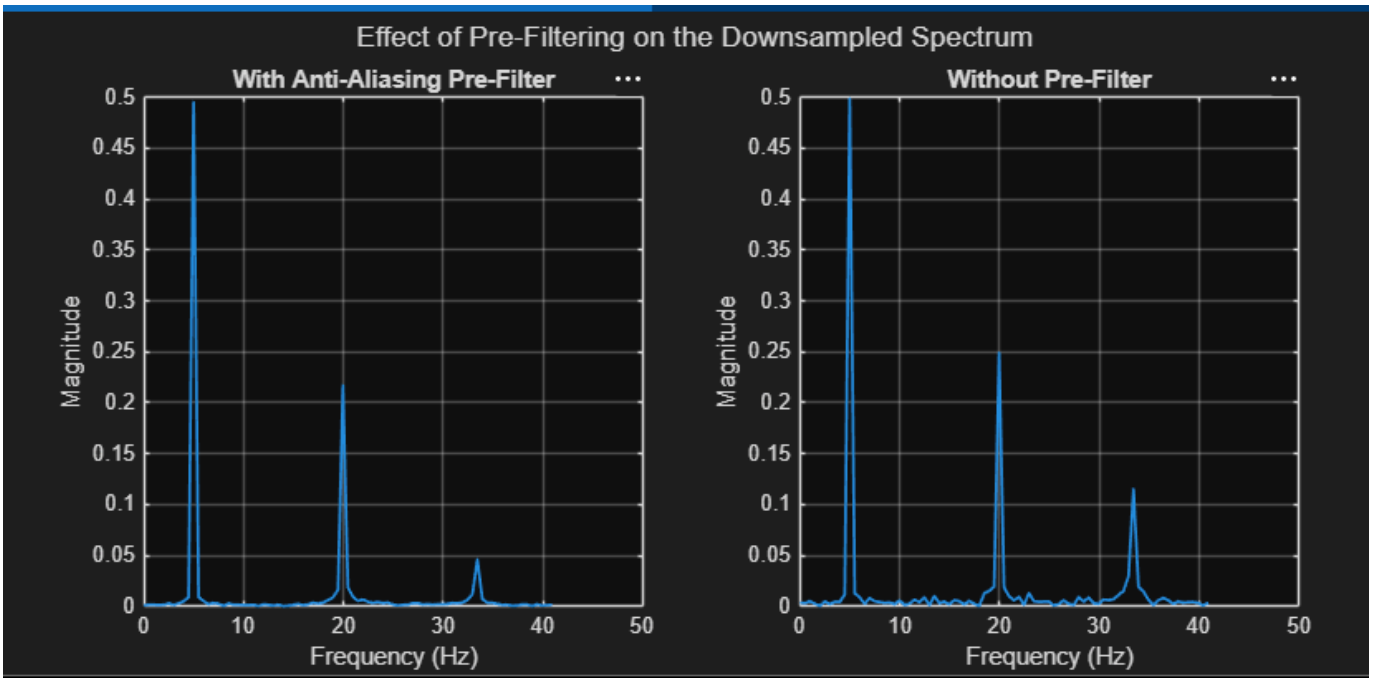


Figure 3: Effect of Pre-Filtering on the Downsampled Spectrum (M = 12). Spectra with and without the anti-aliasing pre-filter; without it, the 50 Hz tone folds down to about 33 Hz.

Experiment 4: noise robustness (M = 4, RMSE).

Noise std	Linear	Cubic spline	Polyphase FIR
0	0.02499	0.0014093	0.0017162
0.05	0.05142	0.044257	0.044174
0.1	0.09303	0.088465	0.088299
0.3	0.26946	0.26534	0.26485

Without noise, spline and polyphase interpolation are about 15 to 18 times more accurate than linear, which shows the true difference in interpolation quality. As noise increases, the error of all methods converges to roughly 0.88 times the noise level, because the noise removed by the filter dominates the error and the choice of interpolator matters less.

Experiment 6: real audio signal.

RMSE:

M	Zero-order hold	Linear	Cubic spline	FFT (sinc)	Polyphase FIR
2	0.01518	0.00842	0.00800	0.00786	0.00789
4	0.03288	0.01246	0.01119	0.01133	0.01125
8	0.06144	0.02890	0.01788	0.01680	0.01679

Correlation:

M	Zero-order hold	Linear	Cubic spline	FFT (sinc)	Polyphase FIR
2	0.98083	0.99415	0.99467	0.99486	0.99482
4	0.90971	0.98827	0.98956	0.98929	0.98946
8	0.64937	0.96662	0.98185	0.98279	0.98276

The ranking matches the synthetic result. The advantage of polyphase FIR over linear grows with M: about 6% lower RMSE at M = 2, 10% at M = 4 and 42% at M = 8. Zero-order hold collapses at M = 8 (correlation 0.649). FFT and polyphase are equivalent in accuracy (RMSE 0.016796 against 0.016792 at M = 8). Unlike the three-tone synthetic signal, real audio has energy spread over many frequencies, so error grows smoothly with M instead of showing a sharp cliff. The absolute RMSE values are small because the audio is normalised to the range ±1, so methods should be compared relative to each other. Only one excerpt of about two seconds was tested.

4.5 Validation and Verification with summary of findings

Validation. The noise-free spline RMSE of 0.0014 shows that the implementation reconstructs a band-limited signal almost exactly. Observed errors follow the theoretical noise floor (0.0424 against 0.043 expected at M = 4) and the Nyquist limit (sharp rise at M = 10 for a 50 Hz tone). FFT and polyphase results agree to about four digits, as expected for two implementations of ideal band-limited interpolation. The same ranking appears on synthetic and real data.

Summary of findings.

1. Spline, FFT and polyphase FIR interpolation are the most accurate and are nearly equivalent; zero-order hold is the least accurate.
2. Polyphase FIR (the Crochiere and Rabiner method) reduces RMSE by about 14% over linear on the synthetic signal at M = 4, and by 42% on real audio at M = 8.
3. The optimal factor is the largest M for which fs/M stays safely above 2·fmax; here M = 8.
4. Pre-filtering helps when content above the new Nyquist frequency exists, but is not always beneficial.
5. Noise dominates the error for noisy references and narrows the gap between methods.

5 Future Enhancement

- Test more real sensor data, such as ECG, accelerometer and temperature recordings, and several audio clips instead of one.
- Report normalised RMSE and compare against the noise-free signal when it is known, to separate interpolation error from removed noise.
- Study the pre-filter design in more detail: filter length, cutoff and edge handling, so that edge transients cannot affect the scored region.
- Add windowed-sinc kernels (such as Lanczos), Kaiser-window FIR designs and IIR zero-phase filters to the comparison.
- Extend to non-uniform and missing-sample reconstruction, fractional rate conversion and multichannel sensors.
- Measure runtime and operation counts for real-time and embedded implementation.

6 Conclusion & Recommendations

This project built a reproducible MATLAB benchmark for reconstructing sparse sensor signals by downsampling and interpolation. Band-limited methods (polyphase FIR and FFT) and cubic spline reconstruct signals far more accurately than zero-order hold and noticeably better than linear interpolation, with the gap widening as the downsampling factor grows. The best factor is limited by the Nyquist condition $f_s/M > 2 \cdot f_{max}$. Noise sets an error floor that no interpolator can beat, and pre-filtering is useful but must be designed with care. Results on synthetic and real data agree.

Recommendations.

- Use polyphase FIR interpolation (or cubic spline for offline work) instead of zero-order hold or linear interpolation when reconstruction quality matters.
- Choose M so that f_s/M is comfortably above $2 \cdot f_{max}$ (about 2.5 times in our tests).
- Use an anti-aliasing filter when out-of-band content or noise is present, and check its effect on the signal edges.
- Use linear interpolation only for small factors or when computation must be minimal.

7 References / Bibliography

1. R. E. Crochiere and L. R. Rabiner, Multirate Digital Signal Processing, Signal Processing, Elsevier, 1983. https://doi.org/10.1016/0165-1684(83)90013-0
2. R. W. Schafer and L. R. Rabiner, “A digital signal processing approach to interpolation,” Proceedings of the IEEE, vol. 61, no. 6, pp. 692–702, 1973.
3. A. V. Oppenheim and R. W. Schafer, Discrete-Time Signal Processing, 3rd ed., Pearson, 2010.
4. V. Panayotov, G. Chen, D. Povey and S. Khudanpur, “LibriSpeech: an ASR corpus based on public domain audio books,” ICASSP 2015, pp. 5206–5210. https://www.openslr.org/12
5. P. Virtanen et al., “SciPy 1.0: fundamental algorithms for scientific computing in Python,” Nature Methods, vol. 17, pp. 261–272, 2020.
6. MathWorks, MATLAB Signal Processing Toolbox documentation: decimate, resample, interpft, interp1.
7. PubMed record 40534299, provided by the guide: https://pubmed.ncbi.nlm.nih.gov/40534299/ [add authors, title and journal].
8. GitHub repository:

   https://github.com/rushikeshborade123/Interpolation-for-Smooth-Reconstruction-of-Sensor-Signals